SPARK

SMART CAMPUS SUSTAINABILITY PLATFORM

Final Product Requirements and Antigravity Build Brief

Version: 1.0

Status: Agreed product structure; implementation-ready scope

Purpose: Primary specification to upload into Antigravity

Date: October 2026

ENERGY MANAGEMENT • FOOD WASTE REDUCTION • CARBON EMISSIONS

DOCUMENT MAP

Use this brief as the source of truth for the product structure, feature scope, technical behavior and acceptance criteria.

01 Build instructions and non-negotiable rules

02 Product overview and navigation

03 Users and access

04 Energy Management

05 Resource Management - Canteen Food Waste Reduction

06 Carbon Emission module

07 OpenCV + YOLO computer-vision integration

08 Core calculations

09 Data model

10 Technical architecture and UI/UX

11 MVP scope and demo data

12 Tests, acceptance criteria and handoff

Core rule: Demo data must never be presented as real measurements or live camera detections.

IMPORTANT INSTRUCTIONS FOR ANTIGRAVITY

Build a working, end-to-end web application named **SPARK** from this specification. Do not stop at a design mockup, a static dashboard, a slide deck, or a plan. Create the project files, implement the stated workflows and calculations, run the application, test the core flows, and fix build/runtime errors before considering the work complete.

1. Inspect the existing repository first. Preserve working configuration and code where possible. If the repository is empty, create the project structure described below.
2. Build the campus-selection page and the three-module campus dashboard exactly as specified.
3. Every visible button, form, filter, tab and primary navigation item must work. Avoid decorative buttons with no action.
4. The application must run in **Demo Mode without any paid API key, real CCTV feed, smart meter or cloud account**. Use clearly labelled synthetic data for the demo.
5. Also implement the real integration path for OpenCV + YOLO person detection. Live camera use must only be labelled LIVE when a configured camera and working CV service are actually available. Otherwise show DEMO, OFFLINE or NOT CONFIGURED accurately.
6. Do not fabricate real campus measurements, real camera detections, energy savings, food-waste reductions or carbon reductions. Label sample values as DEMO / SYNTHETIC.
7. Do not use facial recognition, identify people, or store video by default. Use aggregate occupancy/footfall statistics and short-lived, non-identifying tracks only.
8. The application must have graceful empty, loading, validation, error and offline states. It must be usable on desktop and mobile.
9. Do not require hardware control. SPARK recommends and tracks actions; it must not automatically switch electrical equipment on or off.
10. After implementation, provide concise run instructions, environment setup, the test results, and a list of any integration that still requires real campus credentials or camera access.

1. PRODUCT OVERVIEW

Product name: SPARK **Product type:** Campus sustainability management web application **Primary mission:** Help educational institutions identify and reduce electricity wastage, reduce canteen food waste, and measure campus carbon emissions through data, computer vision, analytics and corrective actions.

SPARK has three top-level modules:

1. **Energy Management** - electricity data, energy-waste detection and corrective actions.
2. **Resource Management** - initially focused on Canteen Food Waste Reduction.
3. **Carbon Emission** - emissions calculation and reduction tracking.

The platform should support multiple campuses. Each campus must have separate buildings, records, alerts, meal sessions, reports and settings. Seed **IET DAVV, Indore** as a selectable demo campus if useful, but label all sample records attached to it as synthetic demo data. Do not imply they are actual IET DAVV measurements. Provide a way for an authorized administrator to add more campus names rather than inventing real campuses.

Product principles

- Software-first and hardware-optional for the basic workflows.
- Manual data entry and CSV import must work without meters or cameras.
- OpenCV + YOLO supports occupancy/footfall detection where camera feeds and model dependencies are available.
- Explainable calculations and alerts are preferred over opaque predictions.
- Every energy, waste or carbon value must communicate whether it is measured, calculated, estimated or simulated.
- Recommendations remain advisory and require a responsible person to review them.

2. NAVIGATION AND INFORMATION ARCHITECTURE

2.1 Campus Selection Page

Route: `/`

The first page is intentionally simple. Show SPARK branding, a short sustainability tagline, and the list of campus names. Each campus entry is selectable. Do not place the three module buttons on this page; they appear after a campus is selected.

Requirements:

- List available campus names.
- Selecting a campus opens that campus's dashboard.
- Show a clear empty state if no campuses are configured.
- Provide an admin-only Add Campus action (name and optional city/location) if admin functionality is implemented.
- Do not display campus metrics before a campus is selected.

2.2 Campus Dashboard

Route: `/campus/:campusId`

Show the selected campus name and exactly **three primary module cards/buttons**:

1. Energy Management
2. Resource Management
3. Carbon Emission

Each card must open its module. A compact current-campus selector and a return-to-campus-list action may appear in the header. The three cards must remain the main choices; do not add unrelated top-level modules.

2.3 Module navigation

Use a clear header that displays the SPARK name, current campus and a way to return to the campus dashboard. Within a module, use a compact side navigation on desktop and a responsive menu on mobile.

Suggested routes:

- `/campus/:campusId/energy`
- `/campus/:campusId/energy/locations`
- `/campus/:campusId/energy/readings`
- `/campus/:campusId/energy/alerts`
- `/campus/:campusId/energy/cameras`
- `/campus/:campusId/resources/food-waste`
- `/campus/:campusId/resources/meal-planner`
- `/campus/:campusId/resources/production`
- `/campus/:campusId/resources/record-waste`
- `/campus/:campusId/resources/inventory`
- `/campus/:campusId/resources/reports`
- `/campus/:campusId/carbon`
- `/campus/:campusId/settings`

Route names may be adjusted to fit the chosen router, but the information architecture and user journeys must remain equivalent.

3. USERS AND ACCESS

Implement a simple, explainable role model. If full authentication cannot be configured for the demo, provide a clearly labelled demo role switcher and keep the authorization logic centralized so real authentication can replace it later.

- **Campus Administrator:** manages campus records, users/roles, settings, tariffs, emission factors and camera configuration; can view all reports.
- **Facilities / Energy Manager:** manages energy inventory and readings, reviews energy alerts, assigns corrective actions, records resolutions and reviews savings.
- **Mess Manager:** creates meal plans, approves preparation suggestions, reviews food-waste reports and manages meal closeout.
- **Kitchen Staff:** views meal plans and enters planned/prepared/served quantities.
- **Waste Logger:** records food-waste quantities, categories and reasons.
- **Campus Reviewer:** read-only dashboards and approved report exports.

Minimum security rules:

- Users should only see data for the selected campus unless their role grants wider access.
- Do not expose camera stream credentials to frontend code or browser logs.
- Keep edit history for corrections to energy and waste records.
- Never allow historical waste records to be silently changed or deleted to improve performance metrics.

4. ENERGY MANAGEMENT

4.1 Energy inventory

Allow authorized users to manage:

- Buildings, floors/zones and rooms (when needed).
- Room/zone type (classroom, laboratory, library, office, hostel, canteen, auditorium, outdoor area, server room, workshop, etc.).
- Equipment category and label (lights, fans, computers, AC/HVAC, pumps, lifts, servers, other).
- Equipment quantity and rated power in watts where known.
- Expected operating schedule and optional occupancy schedule.
- Available electricity meter, meter-to-location association and reading unit.
- Applicable electricity tariff for indicative cost estimates.

Do not assume a campus contains every facility type. Allow the administrator to add or edit the inventory.

4.2 Energy data capture

Support:

- Manual meter-reading entry with reading date/time and unit.
- Electricity bill entry, including billing dates, kWh and bill amount when available.
- CSV upload for readings and historical energy data.
- Equipment-based estimates using rated power, quantity and operating hours.
- Synthetic demo data for demonstrations.
- Source labels: `MEASURED`, `CALCULATED`, `ESTIMATED`, `SIMULATED`.

Validate numeric fields, units, dates, duplicate records and missing values. Show useful validation messages instead of silently dropping bad rows. For CSV import, show a preview, accepted/rejected rows and a clear import result.

4.3 Energy dashboard

Display:

- Recorded consumption in kWh for the selected period.
- Estimated electricity cost where tariff data is available.
- Consumption by building/zone.
- Daily/weekly/monthly trend when the supporting data exists.
- High-use locations for investigation, not a claim that they are necessarily inefficient.
- Potential after-hours or schedule-related waste alerts.
- Alert counts and resolution status.
- Data coverage and source-quality indicators.
- Measured/estimated/simulated badges near the relevant values.

Charts must use actual data in the app state/database. Filters for date range, building and data source must change the displayed results.

4.4 Energy waste detection rules

Start with transparent rule-based detection. Make thresholds configurable and explain every alert.

Rule A - Possible after-hours consumption: Flag a location when available meter data shows usage outside its expected schedule and the value exceeds a configurable baseline/threshold.

Rule B - Unexpectedly high consumption: Compare a location with a comparable historical time window. Do not compare buildings without considering size, usage, schedule or occupancy context.

Rule C - Occupancy and visible equipment condition: If a configured CV feed estimates that a room has been unoccupied for a defined period, combine that observation with the expected schedule and a separately available equipment-state input. If visible light/equipment state is not reliably supported by the installed detector, use a staff-entered state or meter-based evidence; do not pretend person detection alone proves electricity is being wasted.

For every alert, display the location, timestamp/window, triggering evidence, rule name, confidence/source, recommended next action and status. Alert states should include at least `OPEN`, `UNDER REVIEW`, `RESOLVED` and `DISMISSED`.

False-positive safeguards:

- Allow special-event/extended-hours schedule exceptions.
- Consider whether a space may legitimately be empty while equipment is on (security, cleaning, experiments, servers, ventilation, etc.).
- Treat an alert as an investigation prompt, not definitive proof of waste.
- Never control electrical equipment automatically.

4.5 Corrective actions and savings verification

Allow an authorized user to assign an alert/action to a person or department, set priority and due date, add resolution notes, change status and record follow-up evidence.

Savings workflow:

1. Record the baseline period and baseline consumption.
2. Record the corrective action and date.
3. Collect comparable post-intervention readings.
4. Calculate observed difference and percentage change.
5. Label the result as estimated or verified based on evidence quality.

Do not claim verified savings only because an alert was closed. Do not claim causation if occupancy, schedules, weather or operating conditions changed substantially.

5. RESOURCE MANAGEMENT - CANTEEN FOOD WASTE REDUCTION

The first release of Resource Management is the canteen food-waste module. Do not build separate water or paper modules in this MVP; keep the architecture extendable for future resources.

5.1 Meal and menu setup

A mess manager can create a meal session with:

- Date.
- Meal type: breakfast, lunch, snacks or dinner.
- Dining location.
- Menu items.
- Expected diner count.
- Session status: Draft, Open or Closed.

Allow copying a previous menu/meal plan and editing it. Prevent duplicate sessions for the same location/date/meal type unless an administrator explicitly allows multiple services.

5.2 Diner and production records

For each session, capture:

- Expected diners.
- Actual diner count, manually entered or imported as an aggregate from another system if available.
- Planned quantity for each menu item.
- Actual prepared quantity.
- Quantity served where available.
- Safe surplus transferred to an approved destination, where applicable.
- Quantity unit; use kg for comparable waste weighing, with portions/pieces allowed only when clearly labelled.

Flag large differences between planned and actual production for review, but never block urgent or necessary food preparation.

5.3 Waste event logging

Allow multiple waste records for a single meal. Required fields:

- Meal session.
- Food item or ingredient; allow `Mixed / unclassified` only when appropriate.
- Waste category.
- Quantity greater than zero.
- Unit, with kg preferred for waste-weight reporting.
- Reason.
- Disposition, when known.
- Automatically recorded user and timestamp.

Waste categories:

- Preparation scraps.
- Unserved cooked surplus.
- Plate returns.
- Spoiled/expired stock.
- Other.

Reasons:

- Overproduction.
- Low demand.
- Quality/taste.
- Excessive portion.
- Spoilage.

- Handling issue.
- Unknown.

Disposition options:

- Compost/biowaste.
- Approved donation.
- Other approved route.
- General disposal.

Do not count food safely donated or redistributed as discarded waste. Keep preparation waste, unserved surplus, plate returns and spoiled stock separate because they have different causes.

Optional fields: notes and photo. Warn users not to upload identifiable student photos. Confirm when a record is saved and prevent accidental double submission.

5.4 Food Waste dashboard and reports

Display:

- Total recorded food waste in kg.
- Waste per 100 diners.
- Actual diners served.
- Estimated waste cost only when reliable unit-cost data exists.
- Waste by category.
- Top five wasted food items in the selected period.
- Daily/weekly/monthly trend as data supports it.
- Expected vs actual diners.
- Planned vs prepared vs served quantity.
- Missing-data/data-completeness warnings.

Filters: date range, meal type, dining location, food item and waste category. Support CSV export for authorized users.

5.5 Meal closeout

Provide separate `Save Draft` and `Close Meal` actions. Before closeout, show missing diner count, production rows or waste entries. Permit authorized corrections, but preserve a change/audit record stating who changed what, when and why.

5.6 Planning suggestions (advisory)

Use recent comparable meals (same meal type and similar weekday/calendar conditions) to estimate diner demand. If sufficient history is unavailable, ask the manager to enter an estimate. Allow a standard quantity per diner or recipe yield for each item.

Suggested quantity = expected diners x standard quantity per diner.

When the same item repeatedly has high unserved surplus across comparable meals, suggest a cautious adjustment for manager review. Do not automatically reduce quantity based on one unusual session. Track shortage/replenishment events so that waste reduction does not create food unavailability.

5.7 Ingredient stock and expiry (P1 / secondary priority)

Implement if core meal-to-closeout workflow is stable:

- Ingredient master list.
- Ingredient batch and quantity received/remaining.
- Unit, received date, best-before/expiry date and storage location.
- Approaching-expiry alerts.
- Stock-loss records for ingredients discarded because of spoilage/expiry.

5.8 Food waste pilot metrics

- Data completeness: percentage of meal sessions with actual diners, production quantities and waste closeout recorded.

- Waste per 100 diners, split by category.
- Unserved surplus and repeated surplus by item.
- Ingredient stock loss due to spoilage/expiry when inventory is enabled.
- Meal closeout time and missing entries.
- Food availability / shortage incidents.

Use a baseline period before claiming improvement. A 7-14 day baseline is a suggested initial pilot approach, not a guaranteed reduction target.

6. CARBON EMISSION MODULE

Carbon Emission is a separate top-level module on the campus dashboard.

6.1 Electricity-related emissions

Calculate emissions using:

```
Estimated emissions (kg CO2e) = electricity consumption (kWh) x emission factor (kg CO2e/kWh)
```

The emission factor must be configurable and must retain:

- Value and unit.
- Source/reference name or URL.
- Geography/grid applicability.
- Reporting year or effective period.
- Date entered and user who entered it.

Do not invent a real-world emission factor. If no valid factor is configured, show `Emission factor not configured` and do not display a false emissions total. Synthetic demo values, if used for a demo, must be explicitly marked as illustrative and not suitable for real reporting.

6.2 Carbon dashboard

Display, when supported by input data:

- Estimated electricity-related emissions for a selected period.
- Emissions by building/zone.
- Historical emissions trend.
- Baseline vs post-intervention comparison.
- Estimated emissions avoided from energy savings.
- Data source and emission-factor provenance.
- Clear labels for estimated, calculated or simulated figures.

6.3 Food-waste emissions (optional)

Only calculate food-waste-related emissions if an appropriate documented methodology and food-waste emission factor are configured. Do not multiply food waste by the grid electricity emission factor. The module must work even if this optional factor is absent.

6.4 Carbon reports

Allow filtering by date and building/location where supported, and export a summary that includes the emission factor source and limitations. Never state that carbon was reduced unless the input data and comparison method support that conclusion.

7. OPENCV + YOLO COMPUTER-VISION INTEGRATION

SPARK uses computer vision for **two distinct purposes**:

1. **Energy Management:** estimate room occupancy and help identify conditions that may indicate unnecessary energy use.
2. **Canteen Food Waste Reduction:** estimate canteen occupancy and footfall to improve meal-demand planning.

7.1 Technology responsibilities

- **OpenCV:** reads/processes video frames and provides image-processing utilities.
- **YOLO:** detects people in frames.
- **Tracking/crossing logic:** estimates entry/exit counts when camera placement and tracking support it.
- **SPARK backend:** stores time-stamped aggregate results and combines them with schedules, energy data and meal records.

Run computer vision on a local Python service/edge machine connected to a supported camera feed. Do not assume YOLO runs inside every CCTV camera. A camera must support an accessible stream and have permission to be integrated.

7.2 Modes and truthful status

Provide two modes:

- **DEMO MODE:** sample/synthetic occupancy and events. Always usable when hardware is missing.
- **LIVE MODE:** real feed is configured, the CV service is healthy and analysis succeeds.

Display `LIVE` only when these conditions are verified. If unavailable, show `NOT CONFIGURED`, `OFFLINE` or `MODEL UNAVAILABLE` with a useful reason. Never silently substitute demo numbers while claiming they are live detections.

7.3 Minimum CV integration behavior

- Configurable camera records and a camera-health/status panel.
- Backend-only stream credentials; never expose RTSP passwords in the frontend.
- Person detection using a configurable YOLO model path.
- Occupancy estimate for the configured area.
- Optional directional entry/exit counting using a virtual line and suitable tracking.
- Timestamp and camera/location ID for every aggregate event.
- Clear confidence/status indication and last successful analysis time.
- Manual confirmation/correction when an estimate is unreliable.
- Graceful behavior if `ultralytics`, OpenCV, model weights or camera access are unavailable.

Suggested CV service endpoints (names may be adapted):

- `GET /api/cv/health` - service and model availability.
- `POST /api/cv/analyze-frame` - analyze an uploaded/test frame and return person detections/count.
- `GET /api/cameras` - list configured cameras and statuses.
- `POST /api/cameras/{id}/test` - test configured stream connectivity.
- `GET /api/cameras/{id}/status` - health and last analysis information.
- Stream start/stop endpoints may be added if persistent RTSP processing is implemented.

For a single-image analysis endpoint, return structured output such as camera/location ID, timestamp, person count, detections/boxes if preview is enabled, model name, status and confidence metadata. Do not return identity information.

7.4 Energy-specific CV rules

- Detect people in a configured room/zone.
- Estimate an empty-room duration from successive timestamps, not from a single frame.
- Compare the occupancy estimate to the expected schedule.
- If reliable visible equipment-state detection is not available, require meter evidence or a staff-entered equipment state before generating an equipment-wastage alert.
- Person detection does not prove that a light, fan or AC is on; a standard person-detection model must not be presented as an equipment-state classifier.

- Use an optional custom detector/classifier for visible equipment state only if a suitable model is configured and validated. Otherwise show this as unavailable/experimental.

7.5 Canteen-specific CV rules

- Estimate current occupancy in the dining area.
- If supported, count directional crossings at an entrance to estimate footfall during a meal period.
- Aggregate counts by meal window and compare with actual diner count, meals served and waste records.
- Distinguish `current occupancy`, `estimated footfall`, and `confirmed diners` in the interface. They are not interchangeable. Someone may leave and re-enter; counters can miss people or double-count.
- Use occupancy/footfall as a planning signal, not the sole source of truth for meal production.

7.6 Privacy and camera safety

- Do not implement facial recognition, identity matching or criminal identification.
- Prefer on-device/local processing where feasible.
- Store aggregate counts and operational events by default, not continuous video.
- Do not save frames unless a clearly documented, authorized debug option is enabled.
- Restrict camera configuration/status to authorized staff.
- Show a visible privacy note on camera settings pages.

8. CORE CALCULATIONS

Implement calculations in reusable, tested functions. Handle zero, missing and invalid data safely.

Equipment-based energy estimate

```
kWh = power_watts x quantity x operating_hours / 1000
```

Label this as an estimate; rated power is not necessarily actual power.

Indicative electricity cost

```
estimated_cost = kWh x tariff_per_kWh
```

Label it indicative because actual bills may include fixed charges, demand charges, taxes and tariff slabs.

Potential avoidable energy

```
potential_avoidable_kWh = assumed_average_power_kW x suspected_unnecessary_hours
```

This is an estimate until supported by comparable meter readings.

Energy reduction

```
reduction_percent = (baseline_kWh - post_kWh) / baseline_kWh x 100
```

Only calculate when baseline is valid and greater than zero, and periods are comparable.

Food waste per 100 diners

```
waste_per_100_diners_kg = total_waste_kg / actual_diners x 100
```

If actual diners are missing or zero, display `Not available`.

Unserved surplus rate

```
surplus_rate_percent = unserved_cooked_surplus_kg / prepared_cooked_food_kg x 100
```

Use consistent units and a valid non-zero denominator.

Plate waste per diner

```
plate_waste_per_diner_kg = plate_return_waste_kg / actual_diners
```

Estimated waste cost

Sum wasted quantity multiplied by a reliable matching unit cost. If cost data is missing, show `Cost unavailable`, not zero.

Carbon emissions

```
emissions_kg_co2e = electricity_kWh x configured_grid_emission_factor
```

Record the factor provenance with the result.

9. DATA MODEL

Use stable IDs, timestamps, campus association and validation. A relational SQLite database is suitable for a self-contained local MVP; keep the design easy to migrate to PostgreSQL/Supabase later.

Minimum entities:

- **Campus:** id, name, city/location, active status, created_at.
- **User:** id, display name, role, campus scope, active status (use demo identities only in demo mode).
- **Building:** id, campus_id, name, type, optional area/notes.
- **Zone/Room:** id, building_id, name, type, expected schedule.
- **Equipment:** id, zone_id, name, category, quantity, rated_power_w, active status.
- **EnergyReading:** id, campus_id, location/meter reference, timestamp/period, kWh or meter value, source_type, notes, created_by.
- **ElectricityBill:** id, campus_id, period_start, period_end, consumption_kWh, bill_amount, tariff notes, source_type.
- **EnergyAlert:** id, campus_id, location_id, rule, evidence summary, severity, source/confidence, status, created_at.
- **ActionTask:** id, linked alert, assignee/department, priority, due_date, status, resolution note, follow-up data.
- **Camera:** id, campus_id, location_id, name, stream configuration reference held server-side, mode, enabled status, last health result. Never return credentials through public APIs.
- **CameraEvent:** id, camera_id, timestamp, occupancy_count, entry_count, exit_count, mode/source, confidence/status. Do not store identity.
- **MealSession:** id, campus_id, date, meal_type, location, expected_diners, actual_diners, status, created_by.
- **MenuItem:** id, name, category, default_unit, active status.
- **SessionMenuItem:** id, session_id, item_id, planned_qty, prepared_qty, served_qty, safe_surplus_qty, unit.
- **WasteRecord:** id, session_id, item_id/text fallback, category, reason, quantity, unit, disposition, notes, recorded_by, recorded_at.
- **IngredientBatch:** id, campus_id, ingredient_id, received_qty, remaining_qty, unit, received_date, best_before, storage_location.
- **EmissionFactor:** id, factor_type, value, unit, geography/source, source_url/reference, effective_period, entered_by.
- **AuditEvent:** id, campus_id, user_id, entity_type, record_id, action, timestamp, change_summary.

Every relevant reading/report should carry source metadata or link to the source data. Keep campus IDs on all campus-owned records to prevent cross-campus data mixing.

10. RECOMMENDED TECHNICAL ARCHITECTURE

Use a straightforward stack that Antigravity can generate and run locally:

- **Frontend:** React + Vite + TypeScript.
- **Styling:** Tailwind CSS with reusable accessible components.
- **Charts:** Recharts.
- **Icons:** Lucide React.
- **Routing:** React Router.
- **Application API:** Python FastAPI.
- **Local persistence:** SQLite for the MVP; define database access cleanly for future migration.
- **Computer vision:** OpenCV + Ultralytics YOLO in the Python backend/CV module, loaded conditionally so the main application still starts if CV dependencies or model weights are absent.
- **CSV handling:** a well-supported frontend CSV parser for preview/import, with backend validation.
- **Testing:** frontend build/lint and backend unit/API tests where practical.

Keep business calculations in reusable backend/service functions rather than duplicating formulas across components. Frontend charts must consume actual API/database results, not hardcoded values hidden in chart components. Seed data can be synthetic but must be clearly labelled.

Suggested project layout:

```
spark/
  frontend/
    src/
      app/
      components/
      pages/
      features/energy/
      features/food-waste/
      features/carbon/
      services/
      types/
  backend/
    app/
      main.py
      api/
      models/
      schemas/
      services/
      cv/
      seed/
      tests/
    requirements.txt
    requirements-cv.txt
  .env.example
  README.md
```

Exact layout may vary, but keep energy, food-waste, carbon and CV logic modular.

Data flow and API groups

```
Campus user / React UI -> FastAPI API -> validated service logic -> SQLite database
Manual entry / CSV --------------------^                    -> dashboards and reports
Supported CCTV/IP stream -> OpenCV -> YOLO person detection / optional tracking
                          -> aggregate occupancy and footfall events -> FastAPI -> SQLite
Stored aggregates feed energy rules and canteen demand analytics
```

Organize REST operations around these resource groups (exact URL conventions may follow the generated code):

- Campus, building, zone, equipment and operating-schedule CRUD.
- Energy readings, electricity bills, CSV import validation, energy dashboard and alert/action management.
- Meal sessions, menu items, production records, waste records, meal closeout, food-waste dashboard, stock/expiry and CSV export.
- Emission-factor settings and carbon summary/reporting.
- Camera listing/status, CV health, single-frame analysis and stream connectivity tests.

Environment/configuration

Provide `.env.example` and document at least:

- `SPARK_DEMO_MODE=true`
- `DATABASE_URL=sqlite:///./spark.db` or equivalent local SQLite setting
- `CV_ENABLED=false` by default until dependencies/model are ready
- `YOLO_MODEL_PATH=` configurable model path
- Camera stream URLs/credentials stored only in backend configuration or a secured server-side store; never put secrets in the frontend `.env` bundle.

If environment variables or actual camera credentials are absent, Demo Mode must still work.

11. UI / UX REQUIREMENTS

Visual direction: professional campus software with a sustainable green accent, deep navy/charcoal text, neutral backgrounds, readable charts and clear data labels. Avoid excessive gradients, decoration or animation.

- Responsive layout for desktop, tablet and mobile.
- Accessible contrast, clear focus states and labels.
- Reusable cards, tables, forms, filters, status badges, dialogs and empty states.
- Date filters must apply consistently.
- Confirm destructive/corrective actions.
- Use plain language suitable for campus staff.
- Place `DEMO`, `SIMULATED`, `ESTIMATED` and `LIVE` status labels close to the relevant metric.
- Camera pages must show mode, camera health, last analysis time and a clear reason if live analysis is unavailable.
- Food-waste forms must be quick to use on a phone near the kitchen.
- Charts must have legends, units and meaningful empty states.

12. MVP SCOPE AND BUILD ORDER

Must work in the generated MVP

1. Campus selection and campus-specific navigation.
2. Campus dashboard with three primary module cards.
3. Energy: sample building/zone inventory, manual energy reading entry, energy calculation, basic chart/filter, rule-based after-hours/high-usage alerts and action status workflow.
4. Food waste: create meal session, add menu items, record expected/actual diners, planned/prepared/served quantities, submit multiple waste records, close a meal, view calculations/dashboard, filter and export CSV.
5. Carbon: configurable emission factor with source information, electricity-related emissions calculation, date/location summary and clear unavailable state if factor missing.
6. Camera/CV UI with Demo/Live status, camera-health display and demo occupancy results.
7. Python OpenCV + YOLO analysis path with health check and frame-analysis endpoint. Do not claim live integration without a reachable feed and available model.
8. Synthetic seed data, all explicitly labelled, to make the demo useful immediately.
9. Input validation, safe divide-by-zero behavior and source labels.
10. Persistent local data using SQLite where practical.

Next priority after core workflows

- Ingredient batches and expiry alerts.
- More sophisticated tracking/entry-exit counting on live CCTV.
- CSV import preview and detailed rejected-row reporting if not completed initially.
- More report formats and role permissions backed by real authentication.
- Food-waste emissions when a documented factor/method is configured.

Do not make these MVP dependencies

- Paid LLM/AI APIs.
- Facial recognition or identification.
- Automatic switching/control of lights, fans, ACs or other equipment.
- Smart-meter hardware.
- Live CCTV (demo must run without it).
- Complex machine learning for food prediction.
- Automated donation matching or full purchasing/accounting systems.

13. DEMO DATA AND HONEST REPORTING

Create a small but meaningful synthetic campus dataset with several building/zone types and a few energy readings, schedule violations, alerts and actions. Include food-waste sessions across several dates, different waste categories and at least one resolved issue. Include carbon data only when a clearly labelled demo emission factor is provided.

Use labels and/or a persistent demo banner such as `DEMO DATA - NOT REAL CAMPUS MEASUREMENTS`. Never mix a simulated camera count with a real live status. Include at least one legitimate high-usage case that should not raise an alert, to demonstrate that rules are contextual rather than simplistic.

The demo should show a complete connected scenario:

1. Select the demo campus.
2. Open Energy Management and inspect a zone.
3. View a potential after-hours alert with its explanation.
4. Assign and resolve a corrective action.
5. Open Resource Management and review a meal session.
6. Record a waste event and see the totals update.
7. Open Carbon Emission and inspect calculation inputs and factor provenance.
8. Open the camera panel and clearly show whether the displayed counts are demo or live.

14. TESTS AND ACCEPTANCE CRITERIA

Before completion, verify these scenarios:

Navigation and campus separation

- Selecting a campus opens its dashboard.
- Dashboard shows the three required modules.
- Returning to campus selection works.
- Records for one campus do not appear in another campus's views.

Energy

- Equipment estimate calculates kWh correctly.
- Adding a valid reading updates the relevant totals/chart.
- Invalid/duplicate/missing reading data is handled with a clear message.
- Rule-based alerts show their trigger reason and can change status.
- Alert closure alone does not produce verified savings.

Food waste

- A manager can create, edit and close a meal session.
- Production entries save by menu item and unit.
- Multiple waste categories can be added to the same meal.
- Total waste and waste per 100 diners calculate correctly.
- Missing/zero actual diners displays `Not available` for per-diner rates.
- Filtering changes results and CSV export works.
- Corrections create an audit event.
- Suggested quantities remain advisory and manager-editable.

Carbon

- Emissions calculate from configured kWh and factor.
- Factor value, units, source and effective period are visible.
- Missing factor does not produce a fabricated emissions value.

Computer vision

- CV health endpoint reports enabled/disabled/dependency/model state.
- Demo mode is usable without a camera.
- Live mode is not shown unless a configured stream/model is working.
- Frame-analysis result reports person count and metadata without identity recognition.
- Camera outage is surfaced accurately.
- No stream credentials appear in frontend source or API responses.

General

- Application builds and starts using documented commands.
- No major console errors or broken primary controls.
- Forms have loading/success/error states.
- Mobile layout is usable.
- Synthetic and estimated values are labelled.

15. RUN AND HANDOFF REQUIREMENTS

Provide a README that explains:

- Prerequisites and installed versions.
- How to start the frontend and backend locally.

- How to load/reset demo data.
- How to enable the CV dependency set and configure YOLO model weights.
- How to configure a supported CCTV stream without exposing credentials.
- Which parts work in Demo Mode and what requires actual hardware/configuration.
- API health/test commands and basic troubleshooting.

Antigravity should finish by summarizing:

1. What was implemented.
2. Files and modules created.
3. Commands to run the app.
4. Tests/build checks performed and their results.
5. Integrations requiring a real camera stream, model weights, meters or campus data.
6. Known limitations that remain.

Definition of success: SPARK is a running, testable campus sustainability application with the three agreed modules, real calculations and data-entry workflows, an honest Demo/Live camera integration path, and a clear route to connect real campus data later.